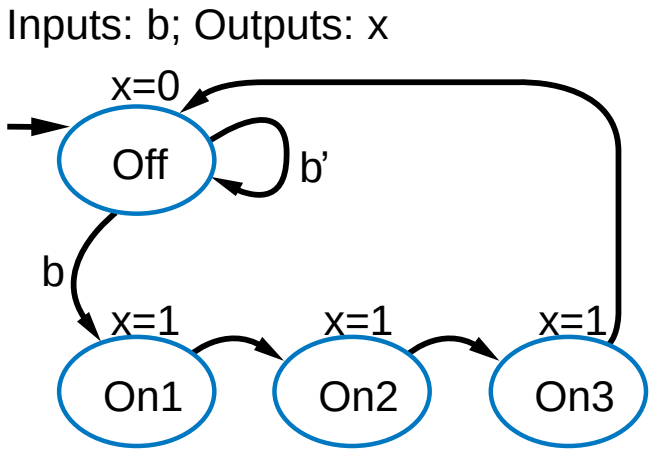
Finite State Machine

FSM Definition

- FSM consists of
  - Set of states
    - Ex: {Off, On1, On2, On3}
  - Set of inputs, set of outputs
    - Ex: Inputs: {b}, Outputs: {x}
  - Initial state
    - Ex: “Off”
  - Set of transitions
    - Each with condition
    - Describes next states
    - Ex: Has 5 transitions
  - Set of actions
    - Sets outputs in each state
    - Ex: x=0, x=1, x=1, and x=1



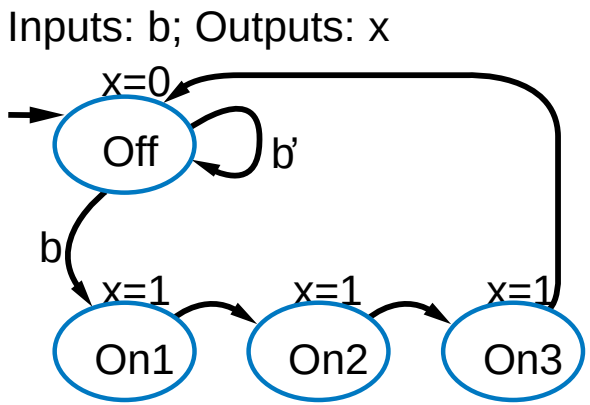
We often draw FSM graphically, known as ***state diagram***

Can also use table (state table), or textual languages

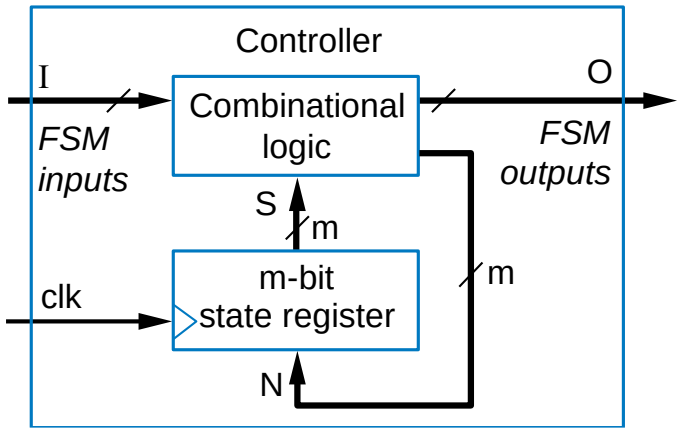
Controller Design

- Converting FSM to sequential circuit
  - Circuit called *controller*
  - Standard controller architecture
    - State register stores encoding of current state
      - e.g., Off:00, On1:01, On2:10, On3:11
    - Combinational logic computes outputs and next state from inputs and current state
    - Rising clock edge takes controller to next state

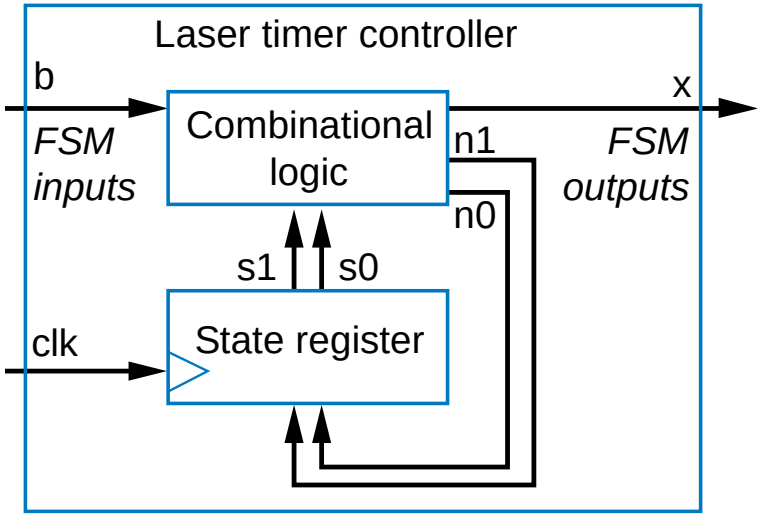
Laser timer FSM



General form



Controller for laser timer FSM

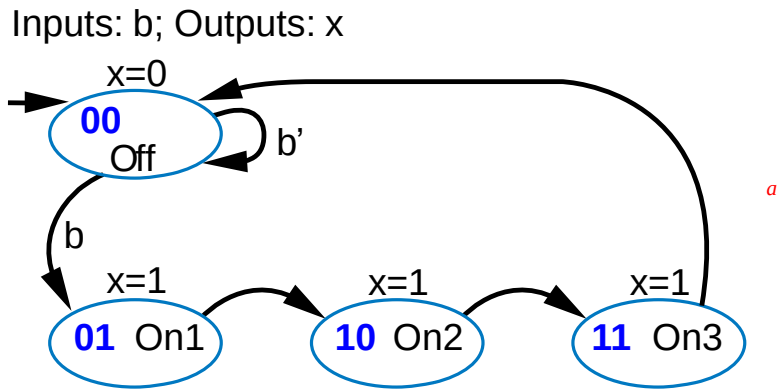


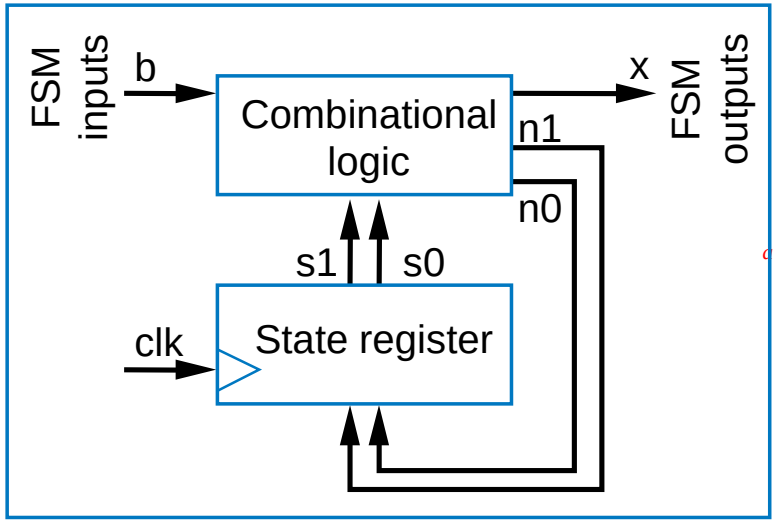
Controller Design Process

Step		Description
Step 1: Capture behavior	**Capture** the FSM	Create an FSM that describes the desired behavior of the controller.
Step 2: Convert to circuit	2A: **Set up** architecture	Use state register of appropriate width and combinational logic. The logic's inputs are the state register bits and the FSM inputs; outputs are next state bits and the FSM outputs.
	2B: **Encode** the states	Assign unique binary number (encoding) to each state. Usually use fewest bits, assign encoding to each state by counting up in binary.
	2C: **Fill in** the truth table	Translate FSM to truth table for combinational logic such that the logic will generate the outputs and next state signals for the given FSM. Ordering the inputs with state bits first makes the correspondence between the table and the FSM clear.
	2D: **Implement** combinational logic	Implement the combinational logic using any method.

Controller Design: Laser Timer Example

- Step 1: Capture the FSM
  - Already done
- Step 2A: Set up architecture
  - 2-bit state register (for 4 states)
  - Input b, output x
  - Next state signals n1, n0
- Step 2B: Encode the states
  - Any encoding with each state unique will work

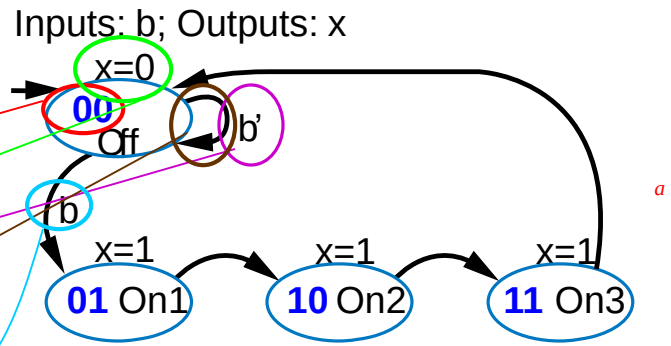


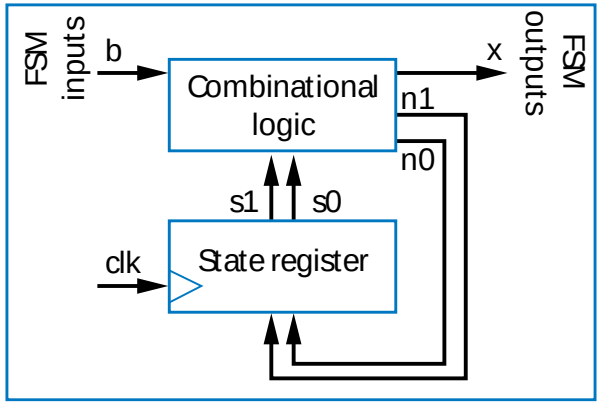


Controller Design: Laser Timer Example (cont)

- Step 2C: Fill in truth table

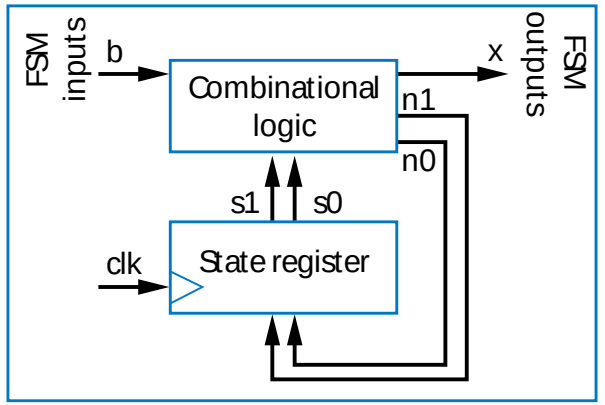
	Inputs			Outputs		
	s1	s0	b	x	n1	n0
Off	0	0	0	0	0	0
	0	0	1	0	0	1
On1	0	1	0	1	1	0
	0	1	1	1	1	0
On2	1	0	0	1	1	1
	1	0	1	1	1	1
On3	1	1	0	1	0	0
	1	1	1	1	0	0





Controller Design: Laser Timer Example (cont)

- Step 2D: Implement combinational logic



	Inputs			Outputs		
	s1	s0	b	x	n1	n0
Off	0	0	0	0	0	0
	0	0	1	0	0	1
On1	0	1	0	1	1	0
	0	1	1	1	1	0
On2	1	0	0	1	1	1
	1	0	1	1	1	1
On3	1	1	0	1	0	0
	1	1	1	1	0	0

x = s1 + s0 (note that x=1 if s1=1 or s0=1)

n1 = s1's0b' + s1's0b + s1s0'b' + s1s0'b

n1 = s1's0 + s1s0'

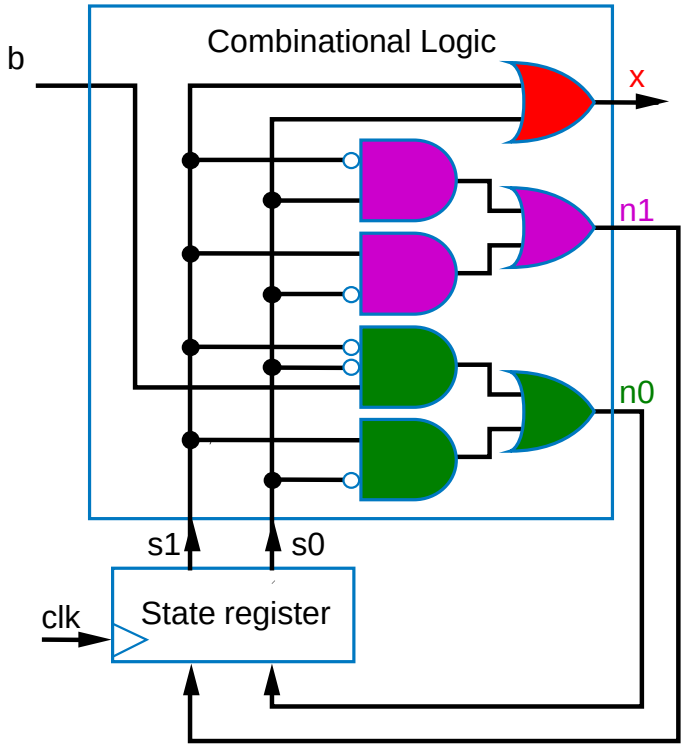
n0 = s1's0'b + s1s0'b' + s1s0'b

n0 = s1's0'b + s1s0'

Controller Design: Laser Timer Example (cont)

- Step 2D: Implement combinational logic (cont)

	Inputs			Outputs		
	s1	s0	b	x	n1	n0
Off	0	0	0	0	0	0
	0	0	1	0	0	1
On1	0	1	0	1	1	0
	0	1	1	1	1	0
On2	1	0	0	1	1	1
	1	0	1	1	1	1
On3	1	1	0	1	0	0
	1	1	1	1	0	0

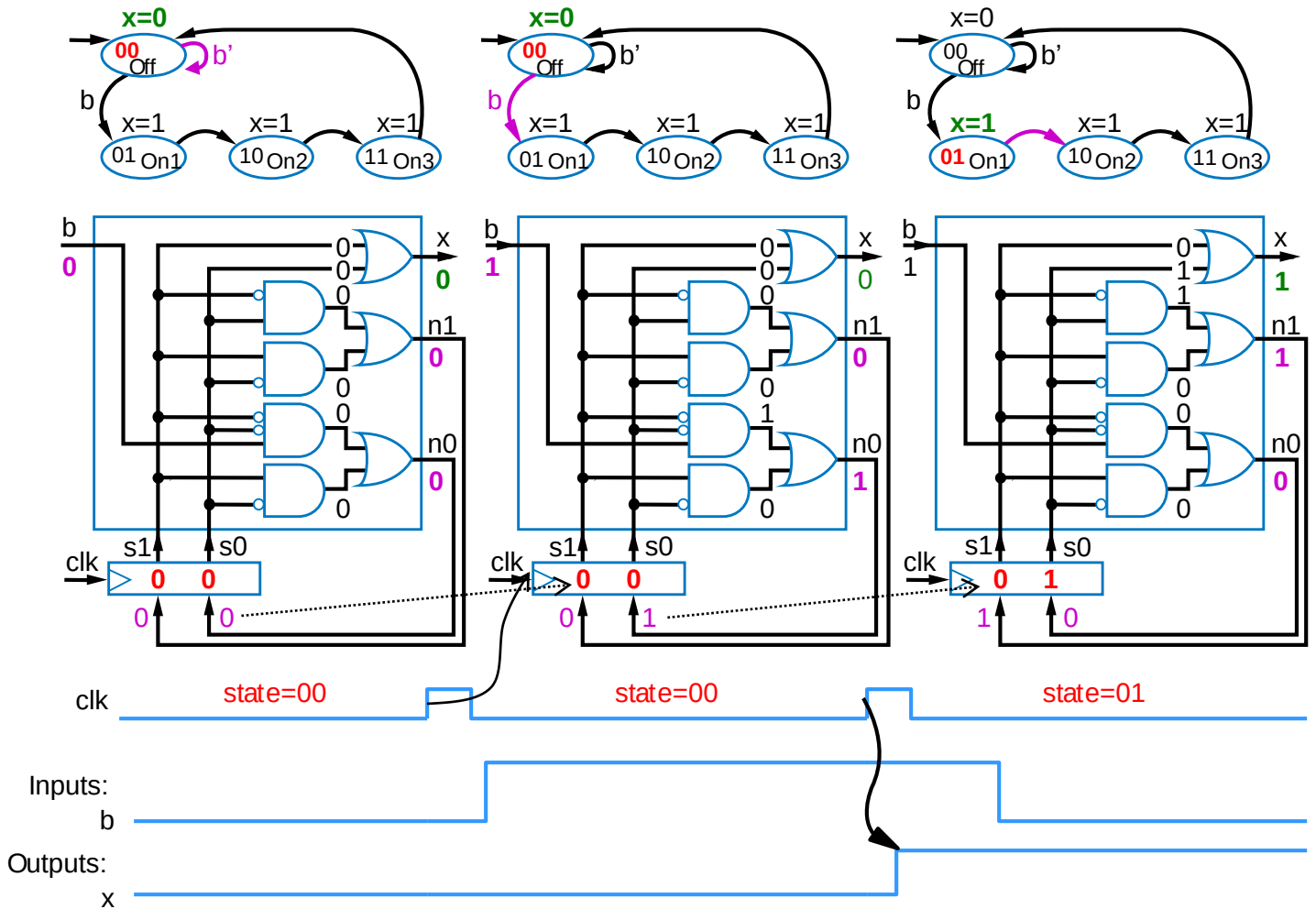


x = s1 + s0

n1 = s1's0 + s1s0'

n0 = s1's0'b + s1s0'

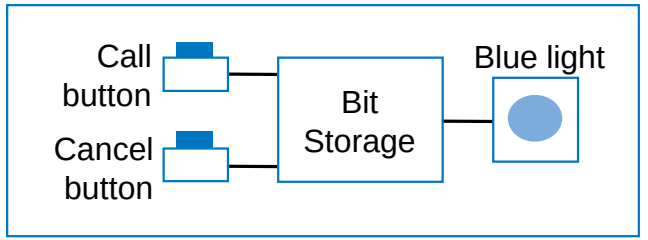
Understanding the Controller's Behavior

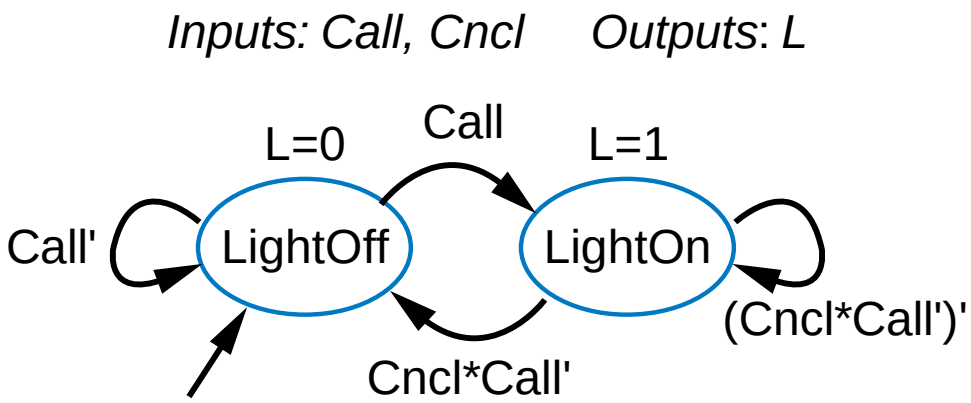


Quiz 6.1

Ex: Earlier Flight-Attendant Call Button

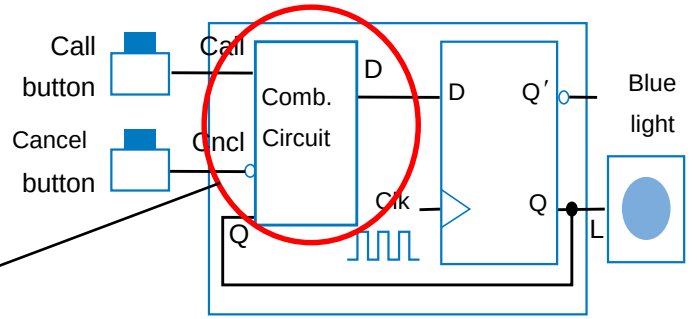
- Previously built using SR latch, then D flip-flop
- Capture desired bit storage behavior using FSM instead
  - Clear and precise description of desired behavior
  - We'll later convert to a circuit



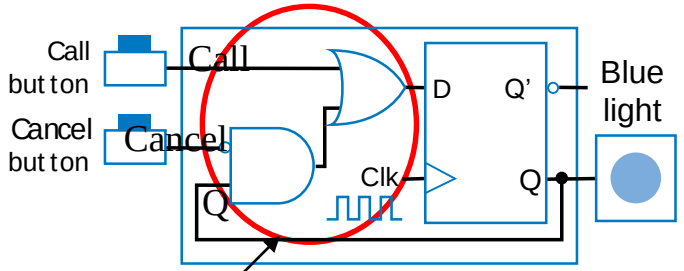


Flight-Attendant Call Button Using D Flip-Flop

- D flip-flop will store bit
- Inputs are Call, Cancel, and present value of D flip-flop, Q
- Truth table shown below

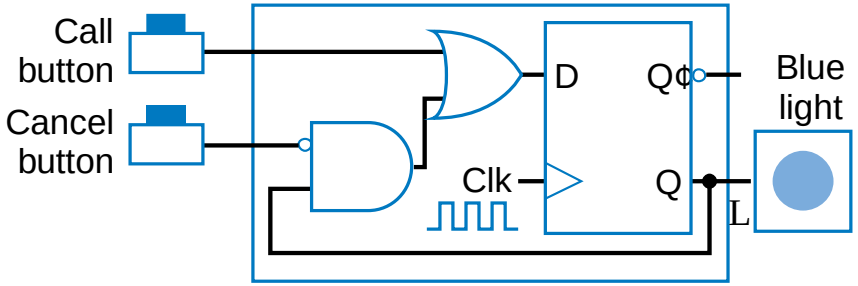


Call	Cancel	Q	D	
0	0	0	0	Preserve value: if Q=0, make D=0; if Q=1, make D=1
0	0	1	1	
0	1	0	0	Cancel -- make D=0
0	1	1	0	
1	0	0	1	Call -- make D=1
1	0	1	1	
1	1	0	1	Let's give priority to Call -- make D=1
1	1	1	1	



Circuit derived from truth table, using Chapter 2 combinational logic design process

Reverse Engin. the D-flip-flop Flight Atten. Call Button



2C: Truth table

	Inputs			Outputs	
	Q	Call	Cncl	D	L
Light Off	0	0	0	0	0
	0	0	1	0	0
	0	1	0	1	0
	0	1	1	1	0
Light On	1	0	0	1	1
	1	0	1	0	1
	1	1	0	1	1
	1	1	1	1	1

2B: (Un)encode states

2D: Circuit to eqns

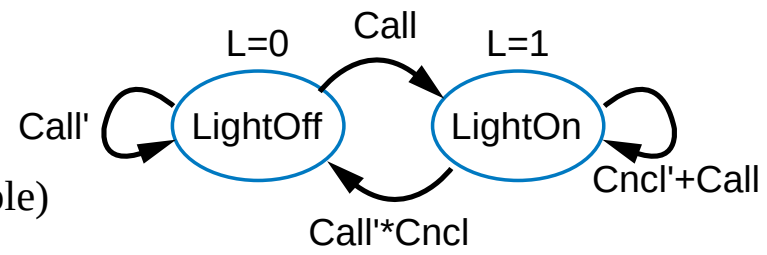
L = Q

D = Cncl'Q + Call (next state)

Don't let the way the circuit is drawn confuse you; the combinational logic is everything outside the register

2A: Set up arch (nothing to do)

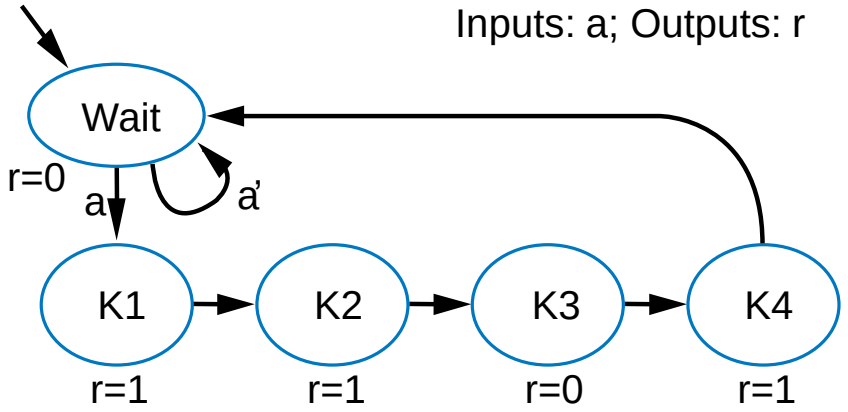
Inputs: Call, Cncl *Outputs : L*



Step 1: FSM (get from table)

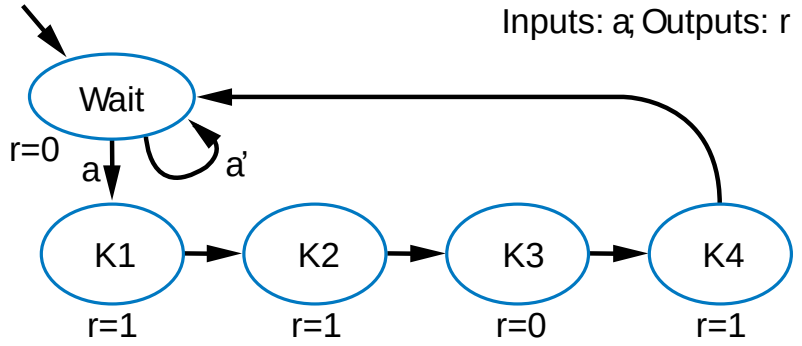
FSM Example: Secure Car Key

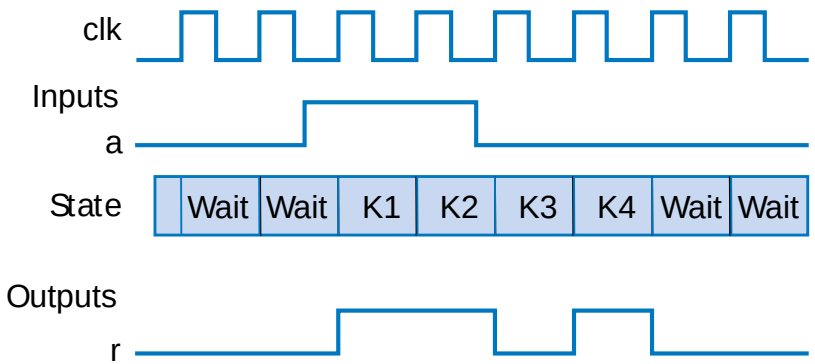
- Many new car keys include tiny computer chip
  - When key turned, car's computer (under engine hood) requests identifier from key
  - Key transmits identifier
    - Else, computer doesn't start car
- FSM
  - Wait until computer requests ID (a=1)
  - Transmit ID (in this case, 1 1 0 1)



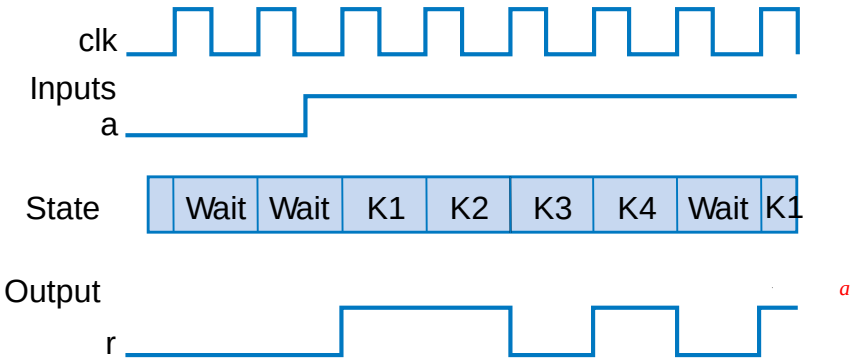
FSM Example: Secure Car Key (cont.)

- Nice feature of FSM
  - Can evaluate output behavior for different input sequence
  - Timing diagrams show states and output values for different input waveforms



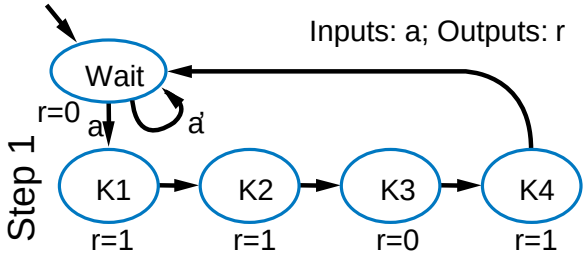


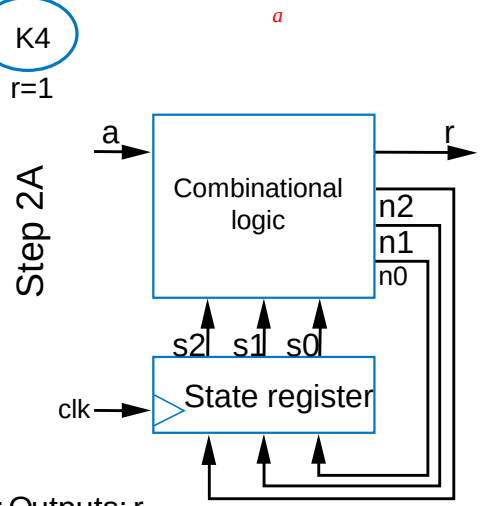
Q: Determine states and r value for given input waveform:

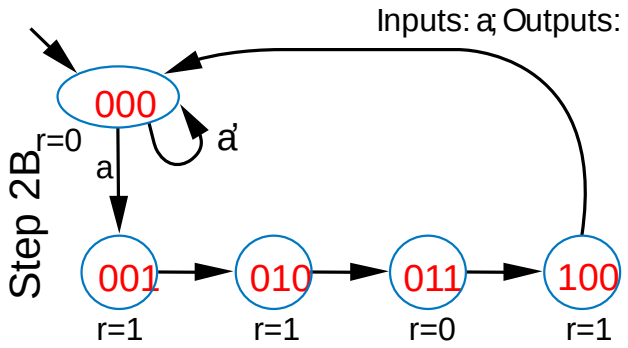


Controller Example: Secure Car Key

- (from earlier example)







	Inputs				Outputs			
	s2	s1	s0	a	r	n2	n1	n0
Wait	0	0	0	0	0	0	0	0
	0	0	0	1	0	0	0	1
K1	0	0	1	0	1	0	1	0
	0	0	1	1	1	0	1	0
K2	0	1	0	0	1	0	1	1
	0	1	0	1	1	0	1	1
K3	0	1	1	0	0	1	0	0
	0	1	1	1	0	1	0	0
K4	1	0	0	0	1	0	0	0
	1	0	0	1	1	0	0	0
Unused	1	0	1	0	0	0	0	0
	1	0	1	1	0	0	0	0
	1	1	0	0	0	0	0	0
	1	1	0	1	0	0	0	0
	1	1	1	0	0	0	0	0
	1	1	1	1	0	0	0	0

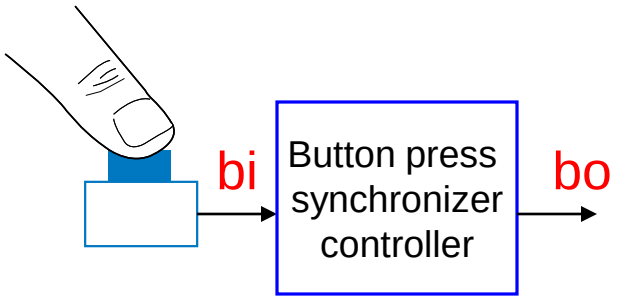
Step 2C

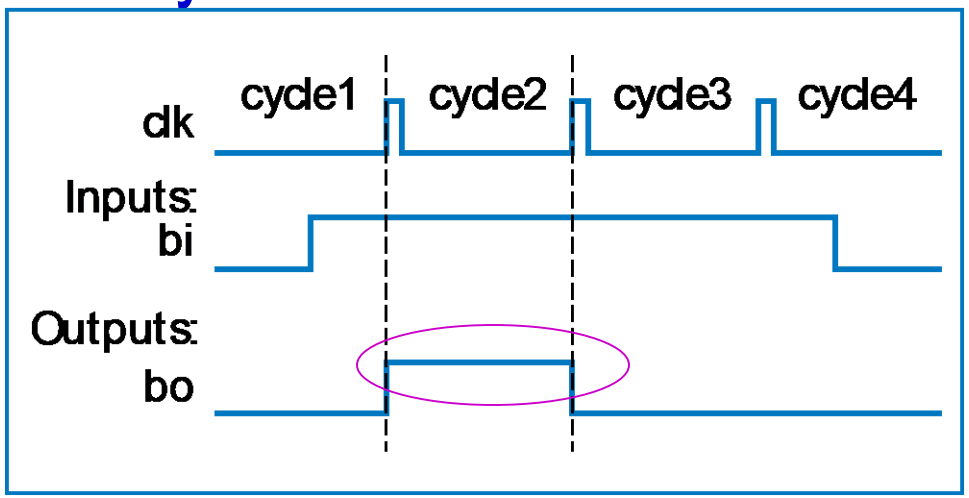
We'll omit Step 2D

How To Capture Desired Behavior as FSM

- *List states*
  - Give meaningful names, show initial state
  - Optionally add some transitions if they help
- *Create transitions*
  - For each state, define all possible transitions leaving that state.
- *Refine the FSM*
  - Execute the FSM mentally and make any needed improvements.

Controller Example:
Button Press Synchronizer

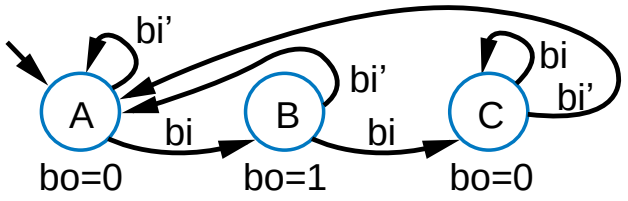




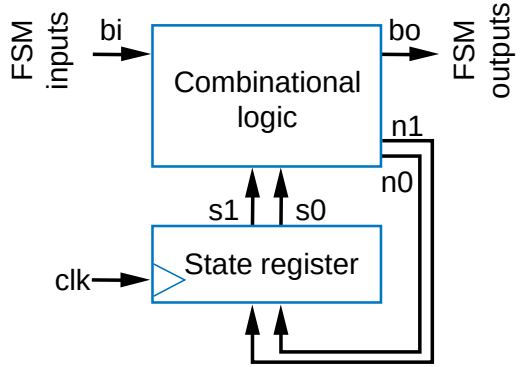
- Want simple sequential circuit that converts button press to single cycle duration, regardless of length of time that button was actually pressed
  - We assumed such an ideal button press signal in earlier example, like the button in the laser timer controller

Controller Example:
Button Press Synchronizer (cont)

FSM inputs: bi; FSM outputs: bo

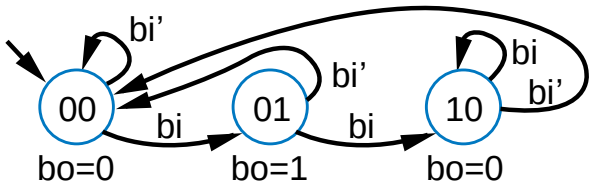


Step 1: Capture FSM



Step 2A: Set up architecture

FSM inputs: bi; FSM outputs: bo



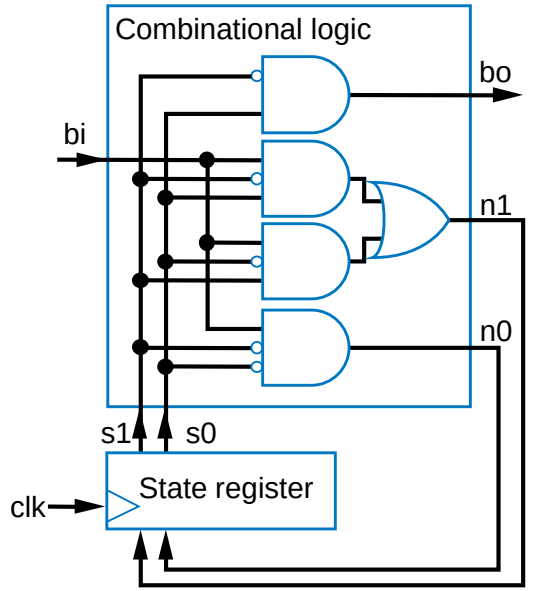
Step 2B: Encode states

Combinational logic

	Inputs			Outputs		
	s1	s0	bi	n1	n0	bo
A	0	0	0	0	0	0
	0	0	1	0	1	0
B	0	1	0	0	0	1
	0	1	1	1	0	1
C	1	0	0	0	0	0
	1	0	1	1	0	0
unused	1	1	0	0	0	0
	1	1	1	0	0	0

Step 2C: Fill in truth table

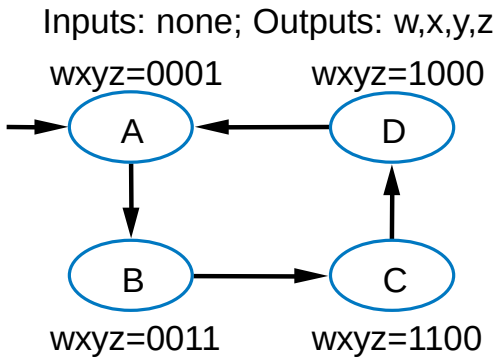
n1 = s1's0bi + s1s0bi
n0 = s1's0'bi
bo = s1's0bi' + s1's0bi = s1s0



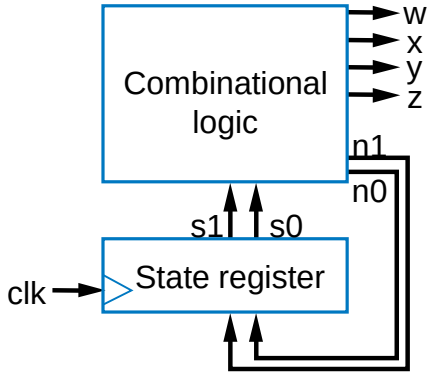
Step 2D: Implement combinational logic

Controller Example: Sequence Generator

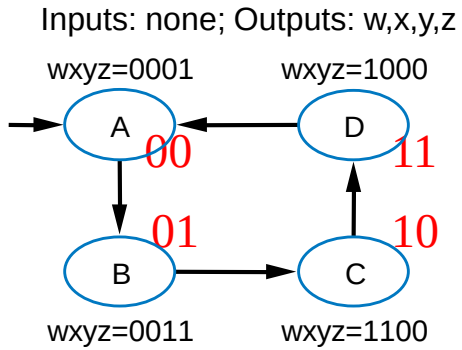
- Want generate sequence 0001, 0011, 1100, 1000, (repeat)
  - Each value for one clock cycle
  - Common, e.g., to create pattern in 4 lights, or control magnets of a “stepper motor”



Step 1: Create FSM



Step 2A: Set up architecture

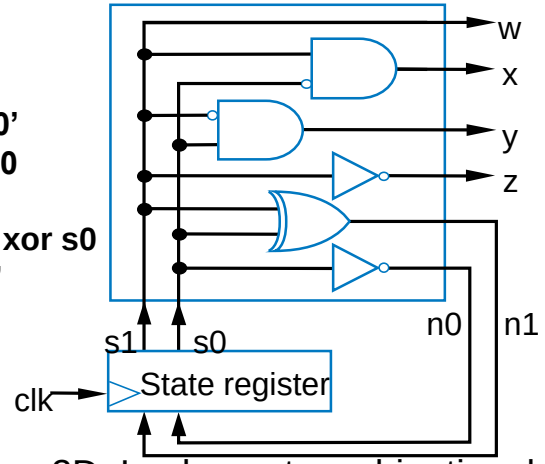


Step 2B: Encode states

Inputs		Outputs					
s1	s0	w	x	y	z	n1	n0
A 0	0	0	0	0	1	0	1
B 0	1	0	0	1	1	1	0
C 1	0	1	1	0	0	1	1
D 1	1	1	0	0	0	0	0

Step 2C: Fill in truth table

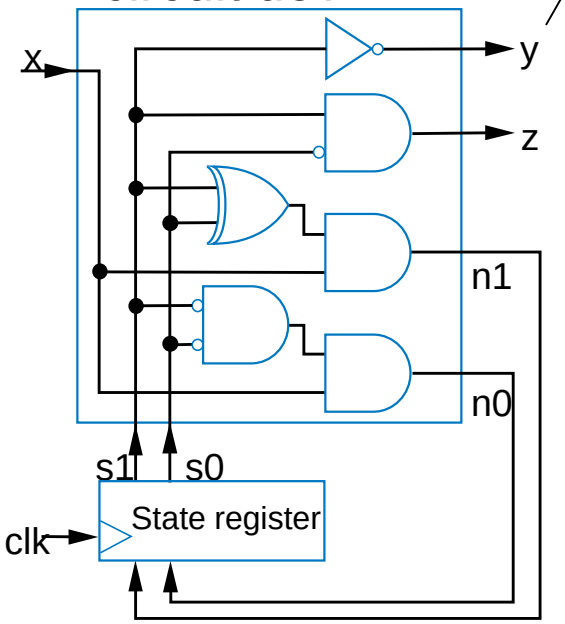
w = s1
x = s1s0’
y = s1’s0
z = s1’
n1 = s1 xor s0
n0 = s0’



Step 2D: Implement combinational logic

Converting a Circuit to FSM (Reverse Engineering)

What does this circuit do?



Work backwards

2D: Circuit to eqns

y=s1'

z = s1s0'

n1=(s1 xor s0)x

n0=(s1'*s0')x

2C: Truth table

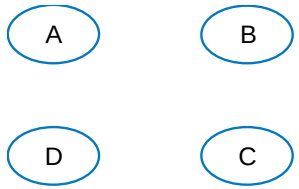
	Inputs			Outputs			
	s1	s0	x	n1	n0	y	z
A	0	0	0	0	0	1	0
	0	0	1	0	1	1	0
B	0	1	0	0	0	1	0
	0	1	1	1	0	1	0
C	1	0	0	0	0	0	1
	1	0	1	1	0	0	1
D	1	1	0	0	0	0	0
	1	1	1	0	0	0	0

2B: (Un)encode states

Pick any state names you want

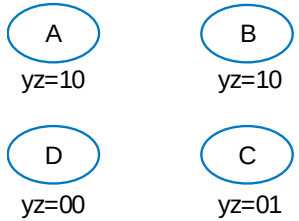
2A: Set up arch – already done

Step 1: FSM (get from table)



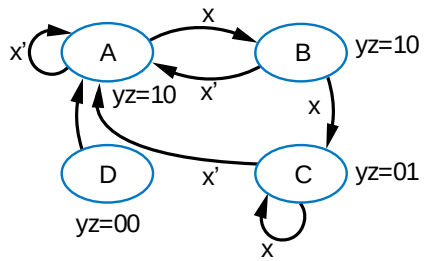
states

Outputs: y, z



states with outputs

Inputs: x; Outputs: y, z

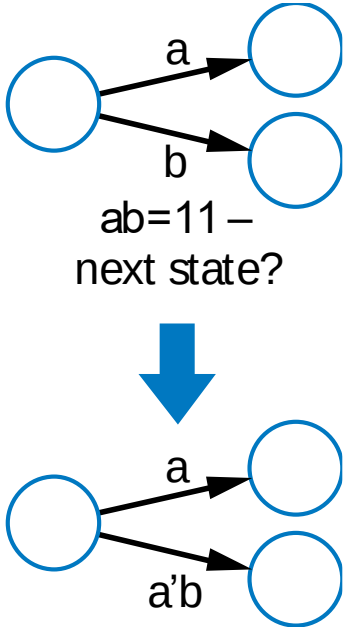


states with outputs and transitions

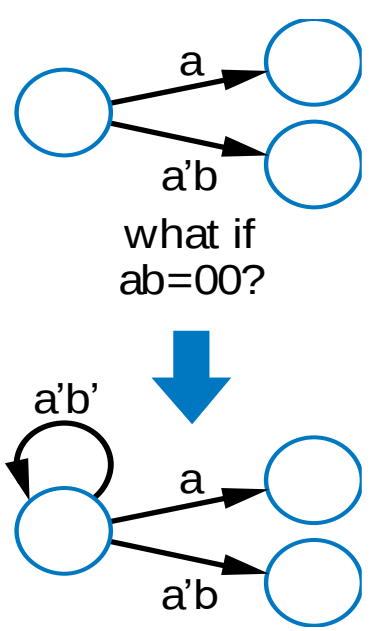
Quiz 6.2

Common Mistakes when Capturing FSMs

- Non-exclusive transitions

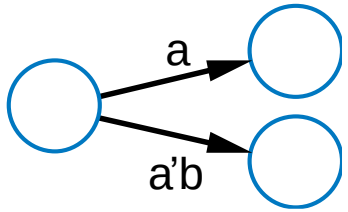


- Incomplete transitions



Verifying Correct Transition Properties

- Can verify using Boolean algebra
  - Only one condition true: AND of each condition pair (for transitions leaving a state) should equal 0 → proves pair can never simultaneously be true
  - One condition true: OR of all conditions of transitions leaving a state) should equal 1 → proves at least one condition must be true
  - Example



Answer:

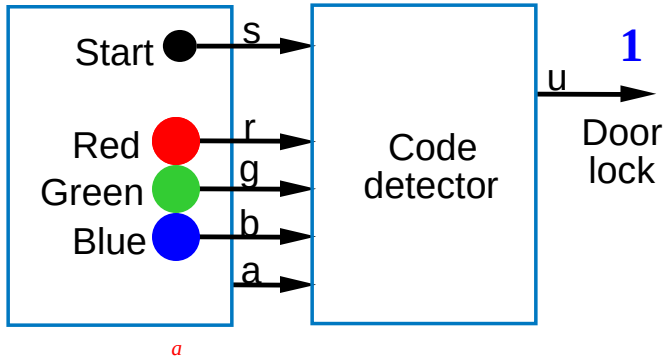
a * a'b
= (a * a') * b
= 0 * b
= 0
OK!

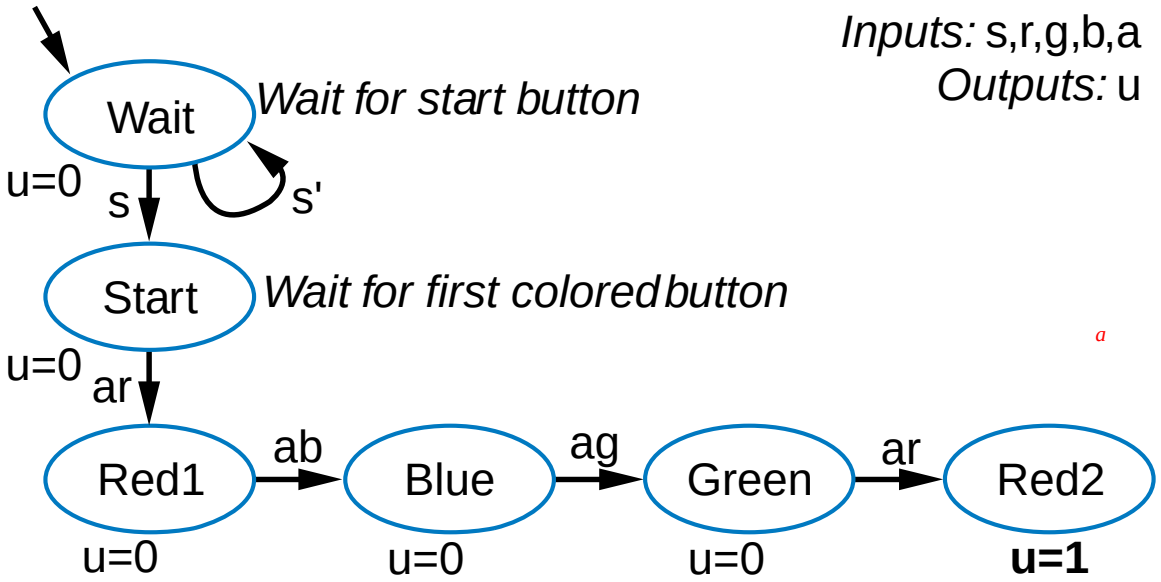
a + a'b
= a*(1+b) + a'b
= a + ab + a'b
= a + (a+a')b
= a + b
Fails! Might not be 1 (i.e., a=0, b=0)

Q: For shown transitions, prove whether:
* Only one condition true (AND of each pair is always 0)
* One condition true (OR of all transitions is always 1)

FSM Capture Example: Code Detector

- Unlock door (u=1) only when buttons pressed in sequence:
  - start, then red, blue, green, red
- Input from each button: *s, r, g, b*
  - Also, output *a* indicates that some colored button pressed
- Capture as FSM
  - ***List states***
    - *Some transitions included*

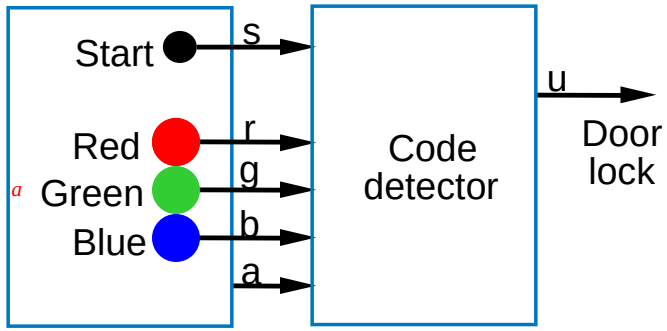




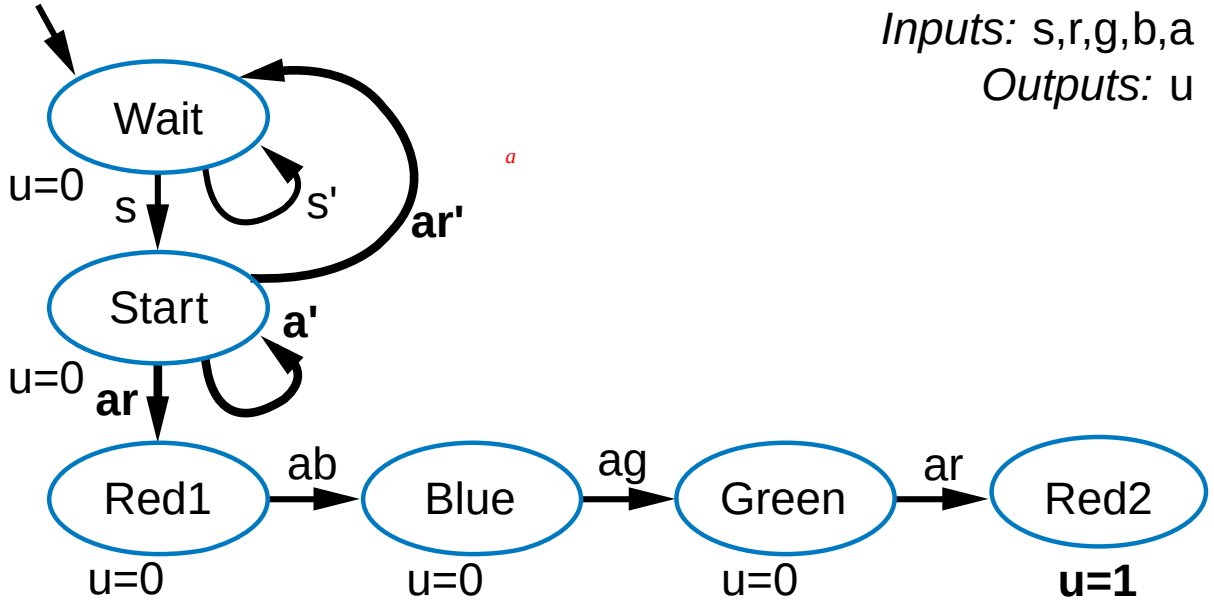
Inputs: s,r,g,b,a
Outputs: u

FSM Capture Example: Code Detector

- Capture as FSM
  - *List states*
  - ***Create transitions***

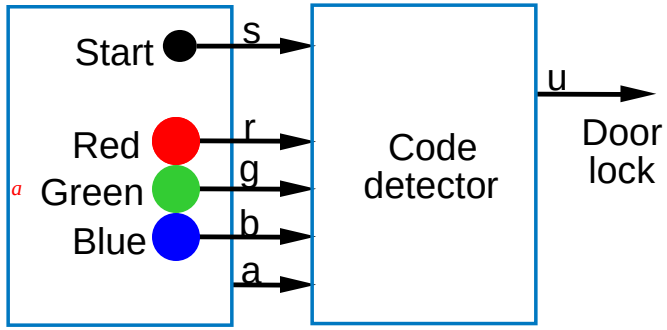


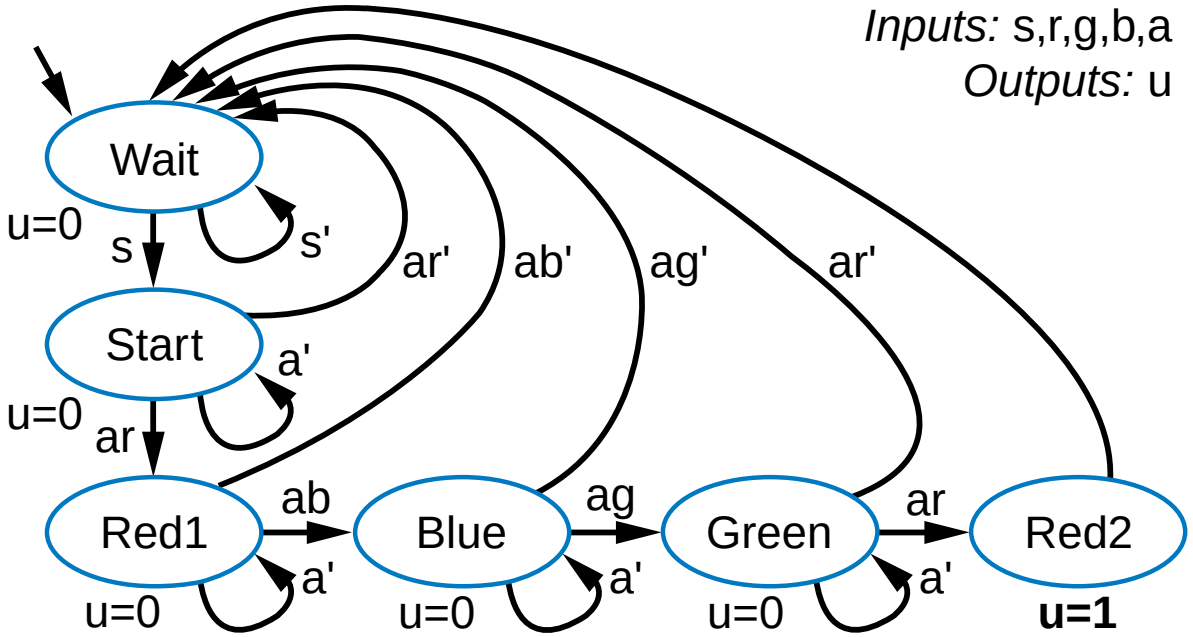
Inputs: s,r,g,b,a
Outputs: u



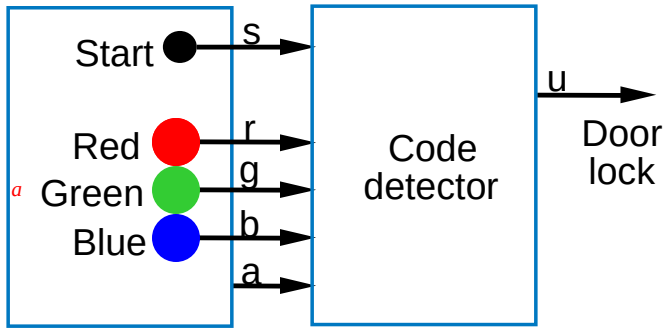
FSM Capture Example: Code Detector

- Capture as FSM
  - *List states*
  - *Create transitions*
    - Repeat for remaining states
  - Refine FSM
    - Mentally execute
    - Works for normal sequence
    - Check unusual cases
    - All colored buttons pressed
      - Door opens!
      - Change conditions: other buttons NOT pressed also

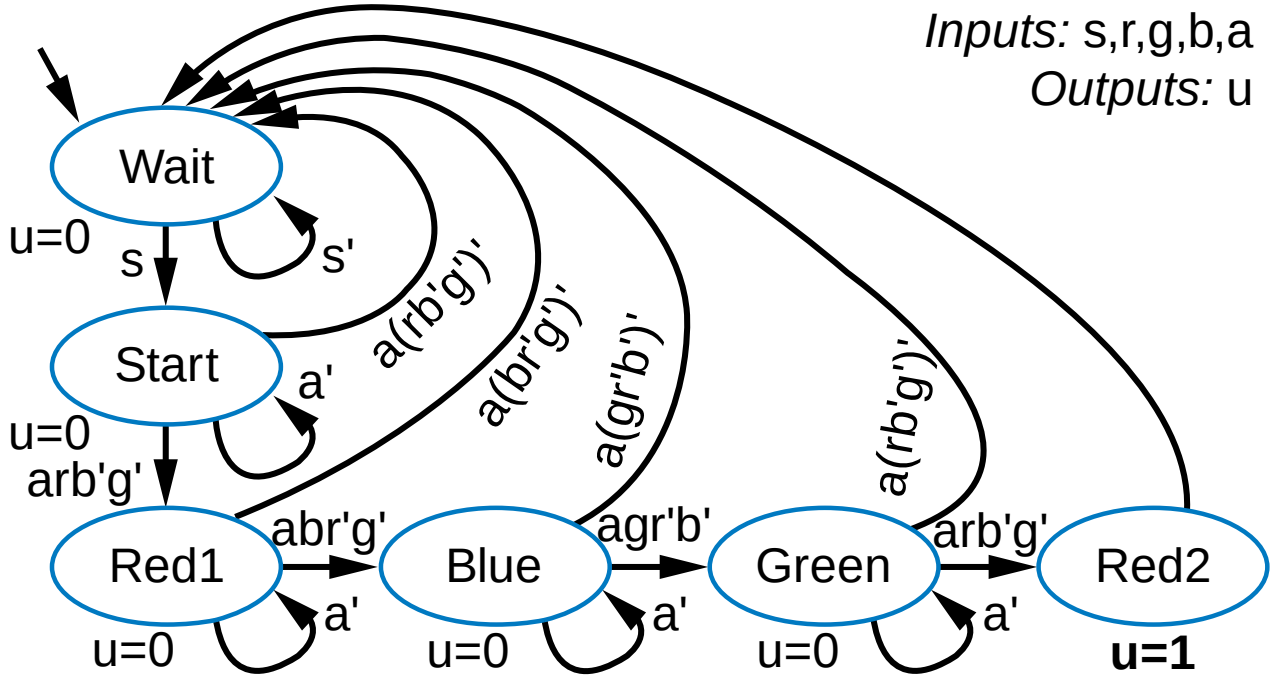




FSM Capture Example: Code Detector

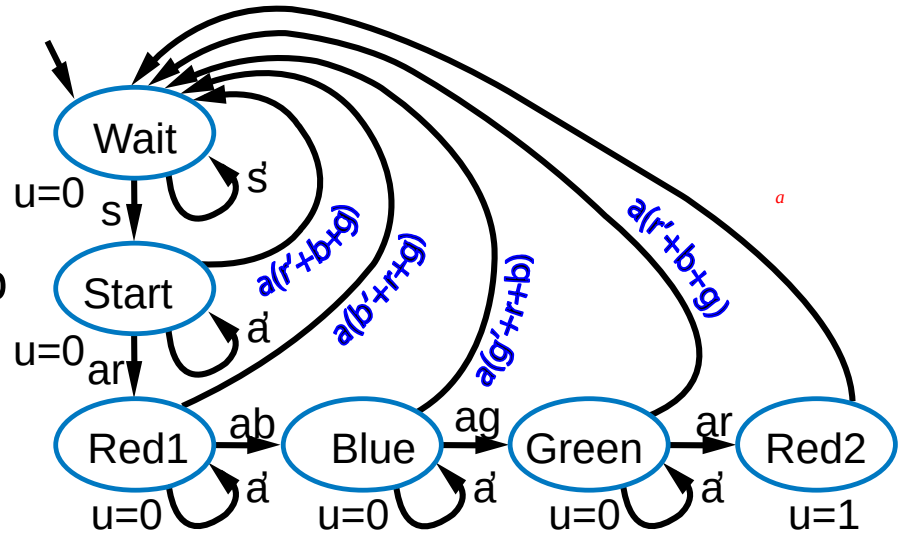


Inputs: s,r,g,b,a
Outputs: u



Verifying transition properties

- Recall code detector FSM
  - We “fixed” a problem with the transition conditions
  - Do the transitions obey the two required transition properties?
    - Consider transitions of state *Start*, and the “only one true” property



ar * a’ = (a*a’)r = 0

a’ * a(r’+b+g) = 0*r = 0

ar * a(r’+b+g)
= (a*a)*r*(r’+b+g) = a*r*(r’+b+g)
= arr’+arb+arg
= 0 + arb+arg
= arb + arg
= ar(b+g)

Intuitively: press red and blue buttons at same time: conditions ar, and a(r’+b+g) will both be true. Which one should be taken?

Q: How to solve?

A: ar should be arb’g’ (likewise for ab, ag, ar)

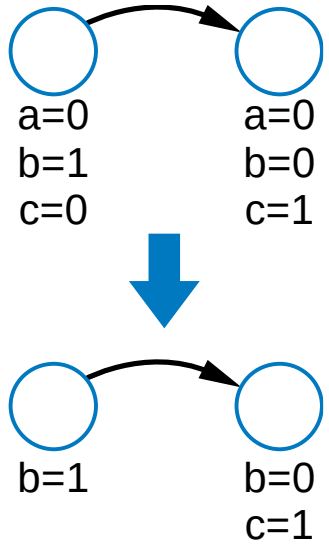
Fails! Means that two of Start’s transitions could be true

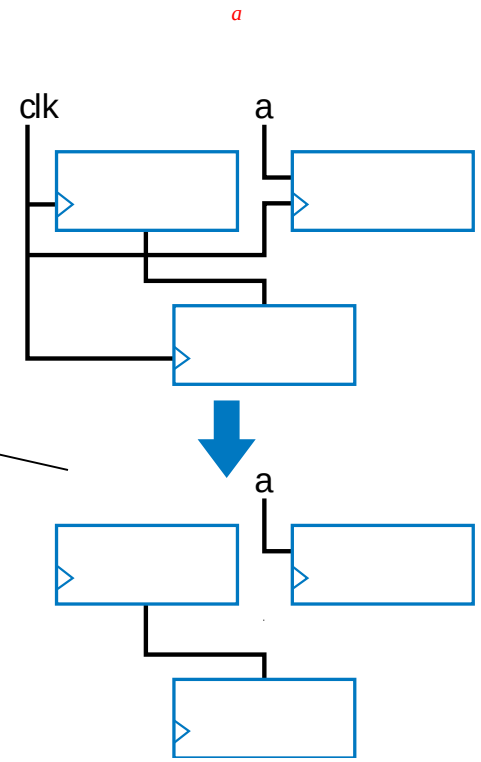
Note: As evidence the pitfall is common, we admit the mistake was not initially intentional. A reviewer of an earlier edition of the book caught it.

Quiz 6.3

Simplifying Notations

- FSMs
  - Assume unassigned output implicitly assigned 0
- Sequential circuits
  - Assume unconnected clock inputs connected to same external clock



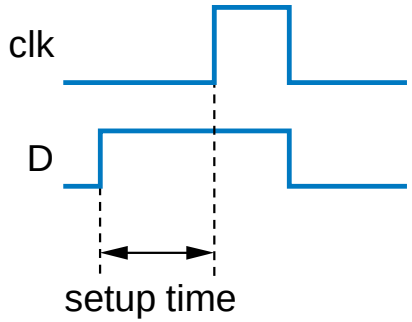


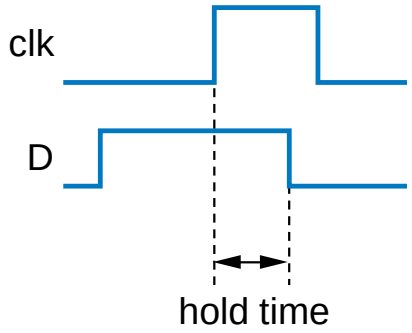
Mathematical Formalisms

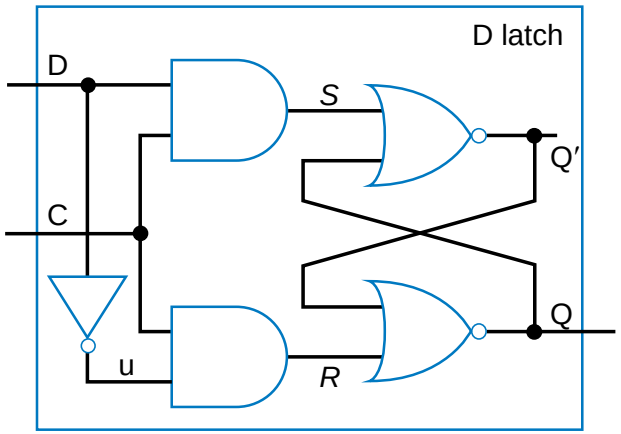
- Two formalisms to capture behavior thus far
  - Boolean equations for combinational circuit design
  - FSMs for sequential circuit design
- Not *necessary*
  - But tremendously *beneficial*
    - Structured methodology
    - Correct circuits
    - Automated design, automated verification, many more advantages

More on Flip-Flops and Controllers

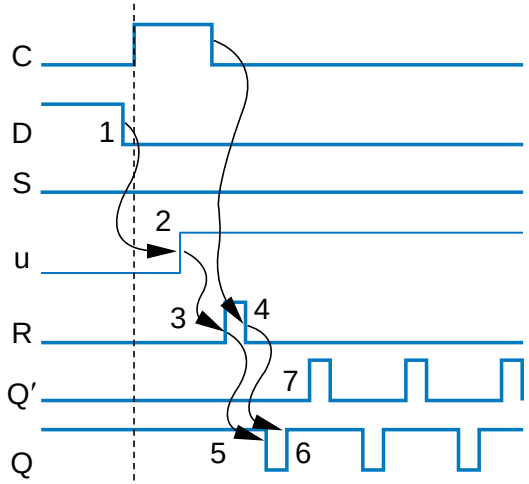
- Non-ideal flip-flop behavior
  - Can't change flip-flop input too close to clock edge
  - Setup time: time D must be stable *before* edge
    - Else, stable value not present at internal latch
  - Hold time: time D must be held stable *after* edge
    - Else, new value doesn't have time to loop around and stabilize in internal latch







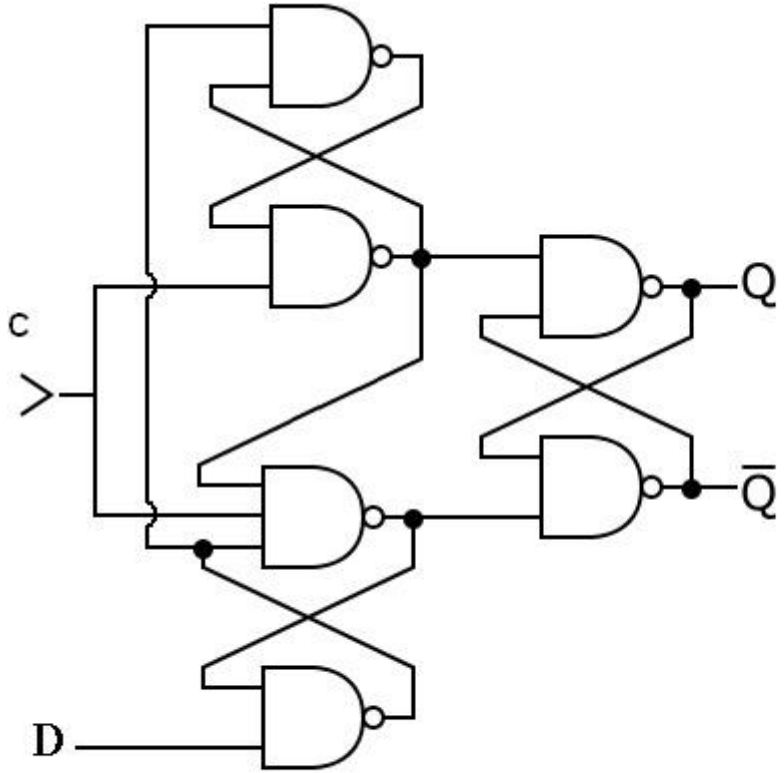
Setup time violation



Leads to oscillation!

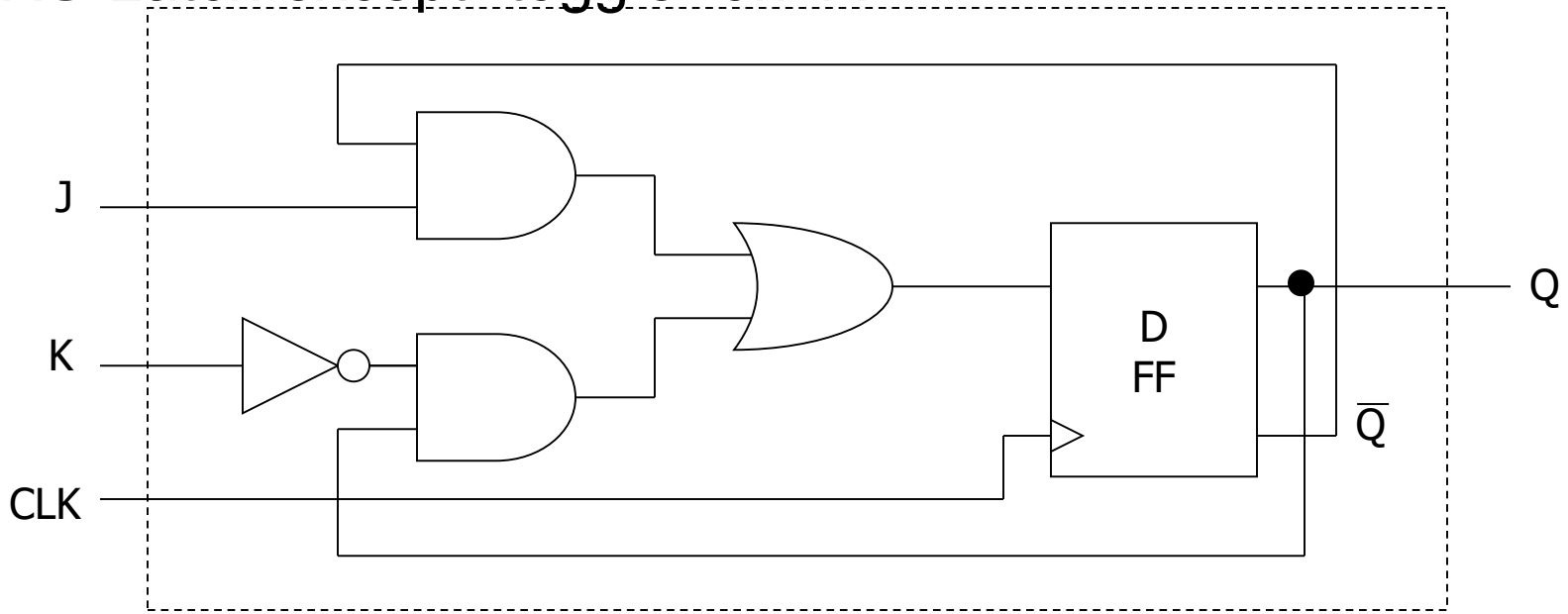
D-FF

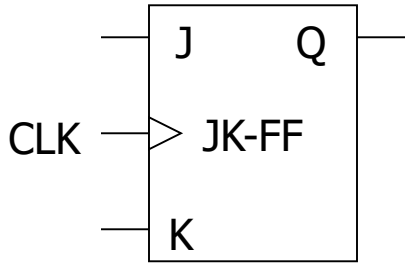
- Please analyze the following circuit proving that it is a positive-edge triggered D-flipflop.



JK Flip-Flop from D-FF

- Same as RS-Latch except “toggle” on 11

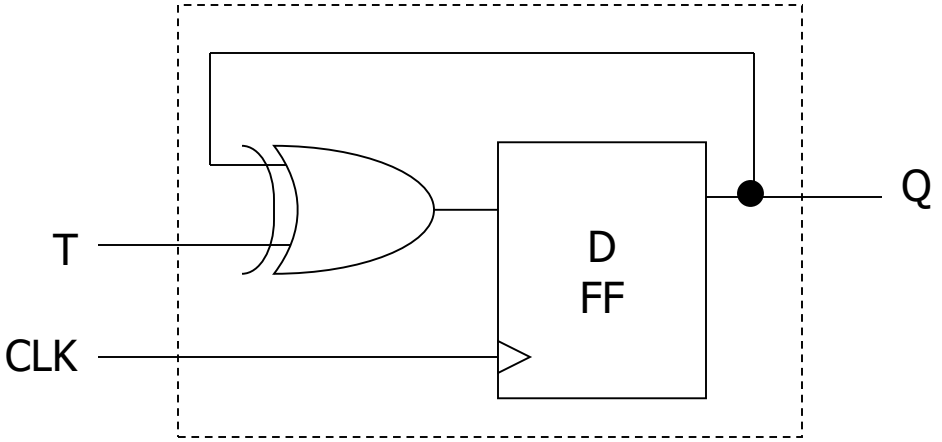


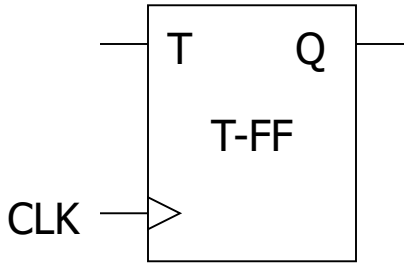


CLK	J K	Q
0	X X	No change
1	0 0	No change
1	0 1	0
1	1 0	1
1	1 1	**Toggle**

Toggle Flip-Flop from D-FF

- Toggles stored value if T = 1 when CLK is high



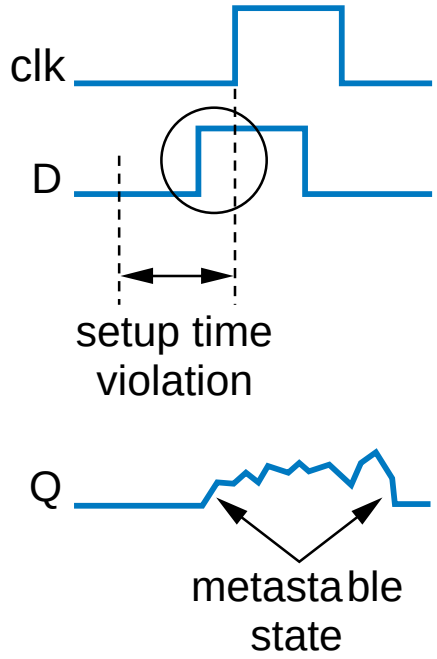


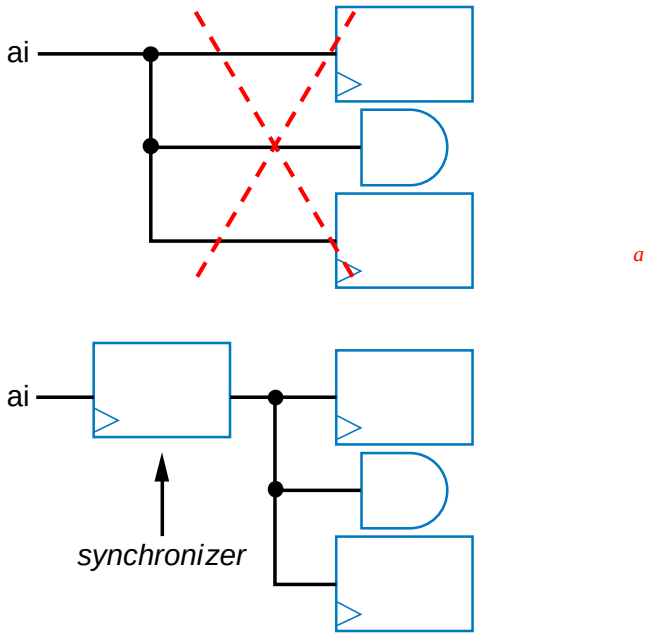
CLK	T	Q
0	X	No change
1	0	No change
1	1	**Toggle**

Quiz 6.4

Metastability(亞穩態)

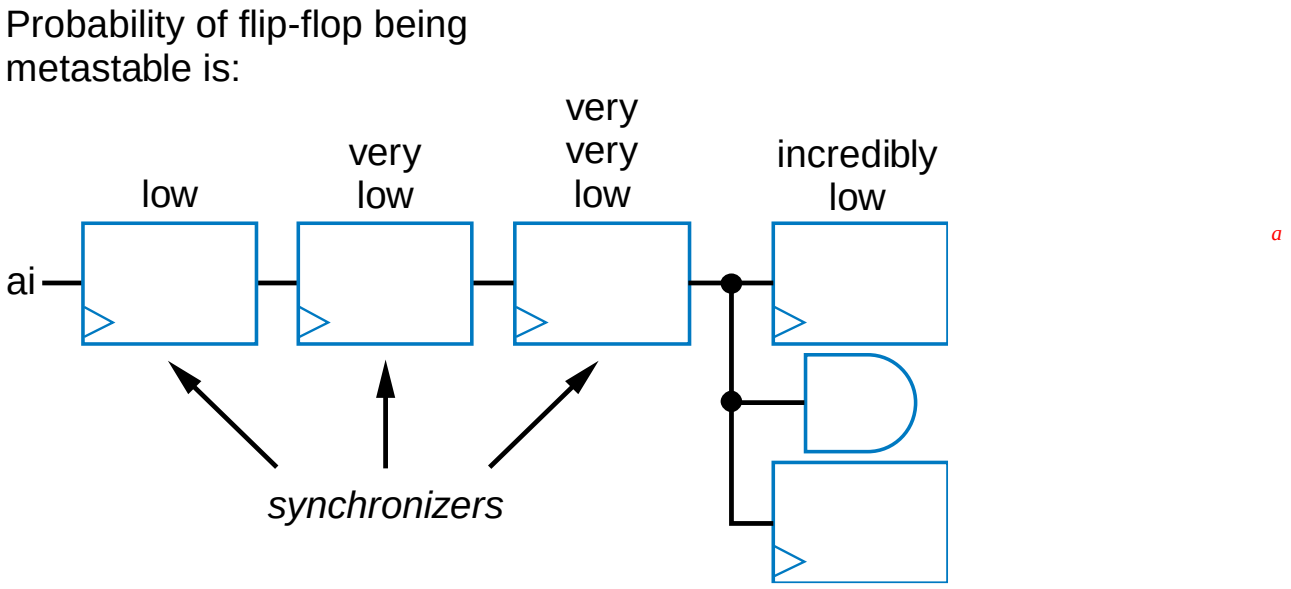
- Violating setup/hold time can lead to bad situation
  - Metastable state: Any flip-flop state other than stable 1 or 0
    - Eventually settles to either, but we don't know which
  - For internal circuits, we can make sure to observe setup time
  - But what if input is from external (asynchronous) source, e.g., button press?
- Partial solution
  - Insert synchronizer flip-flop for asynchronous input
    - Special flip-flop with very small setup/hold time





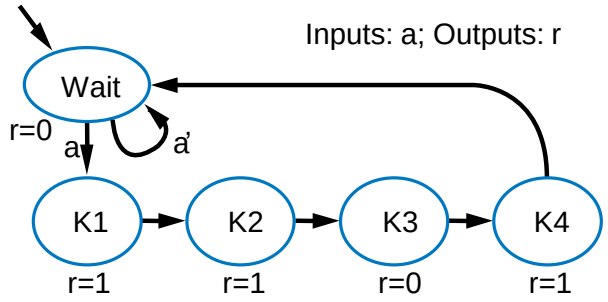
Metastability

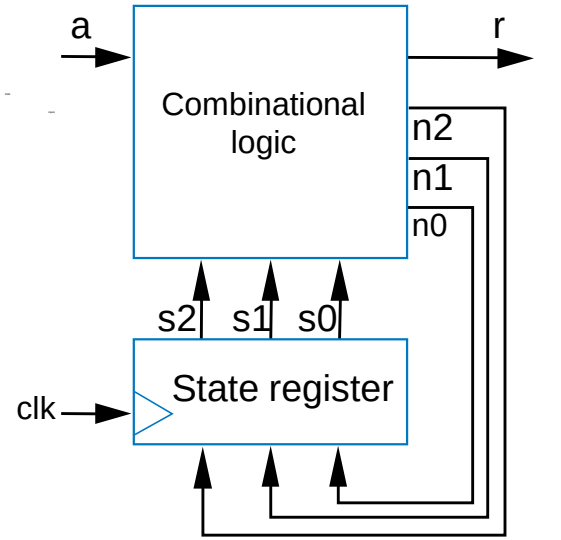
- Synchronizer flip-flop doesn't completely prevent metastability
  - But reduces probability of metastability in dozens/hundreds of internal flip-flops storing important values
  - Adding more synchronizer flip-flops further reduces probability
    - First ff likely stable before next clock; second ff very unlikely to have setup time violated
  - Drawback: Change on input is delayed to internal flip-flops
    - By three clock cycles in below circuit



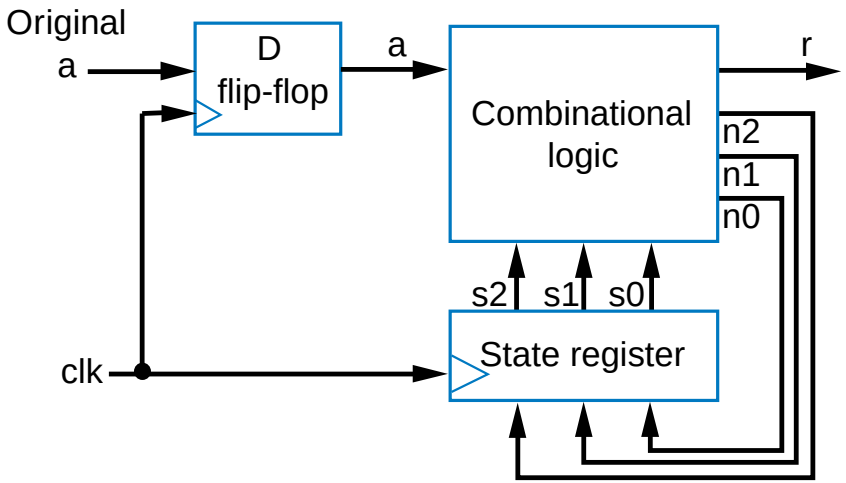
Example of Reducing Metastability Probability

- Recall earlier secure car key controller



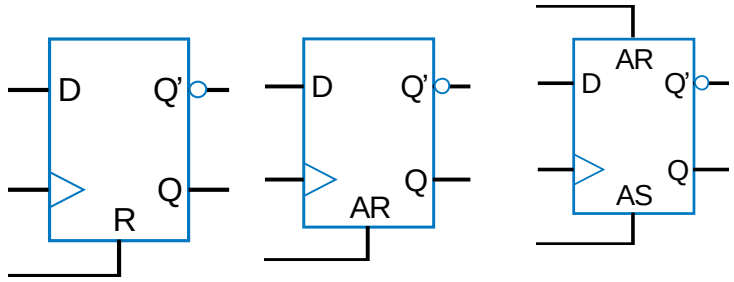


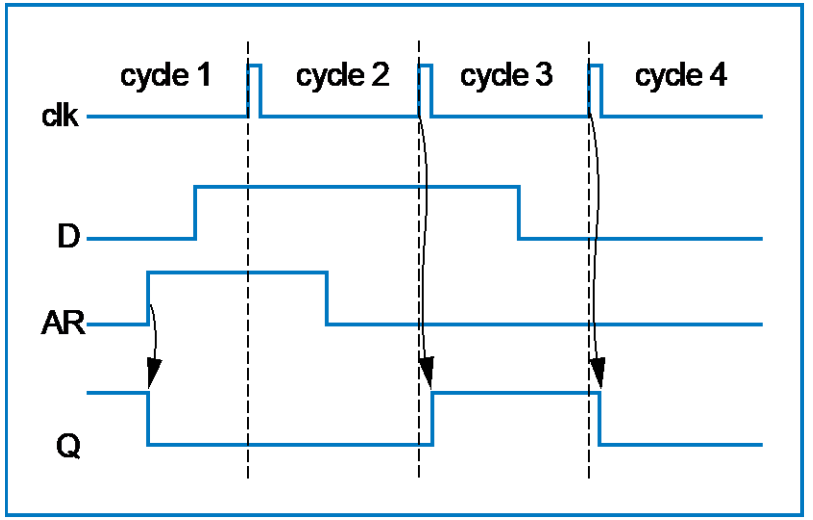
Adding synchronizer flip-flop reduces metastability probability in state register, at expense of 1 cycle delay



Flip-Flop Set and Reset Inputs

- Some flip-flops have additional reset/set inputs
  - Synchronous
    - Synch. reset: Clears Q to 0 on next clock edge
    - Synch. set: Sets Q to 1 on next clock edge
    - Have priority over D input
  - Asynchronous
    - Asynch. reset: Clear Q to 0, independently of clock
      - Example timing diagram shown
    - Asynch. set: set Q to 1, indep. of clock

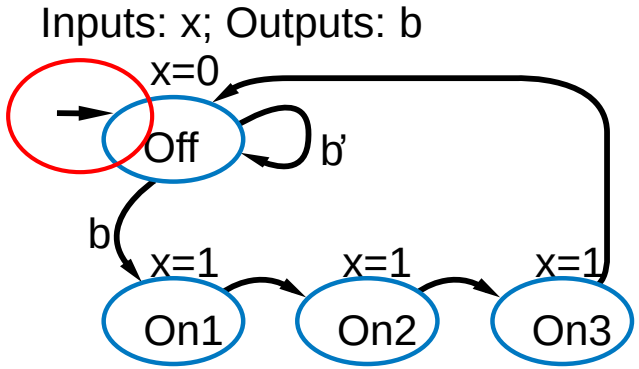


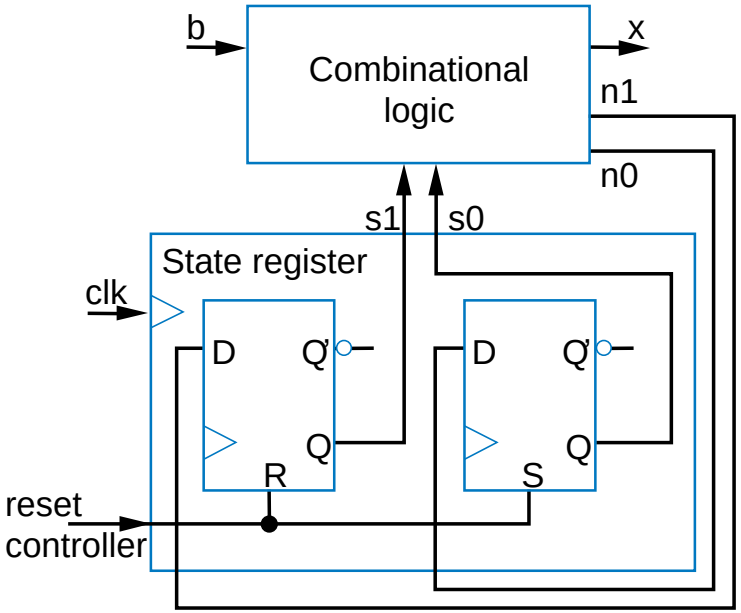


Initial State of a Controller

- All our FSMs had initial state
  - But our sequential circuits did not
  - Can accomplish using flip-flops with reset/set inputs
    - Shown circuit initializes flip-flops to 01
  - Designer must ensure reset-controller input is 1 during power up of circuit
    - By electronic circuit design

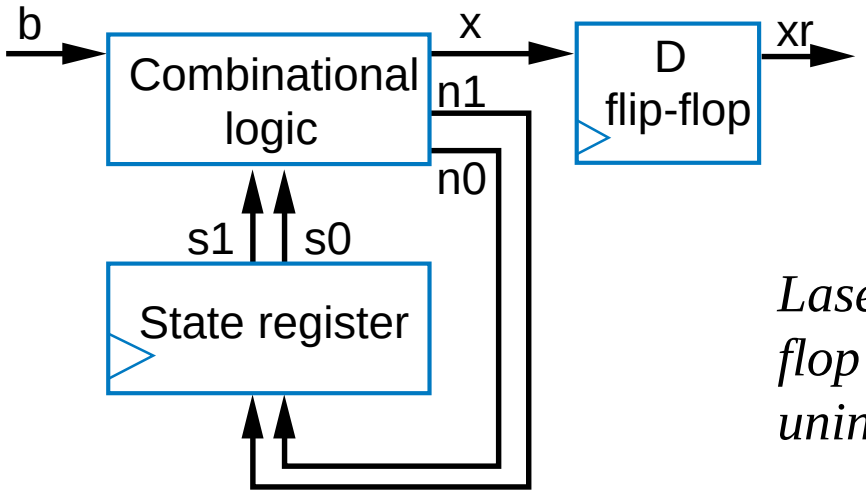
Controller with reset to initial state 01 (assuming state Off was encoded as 01).





Glitching

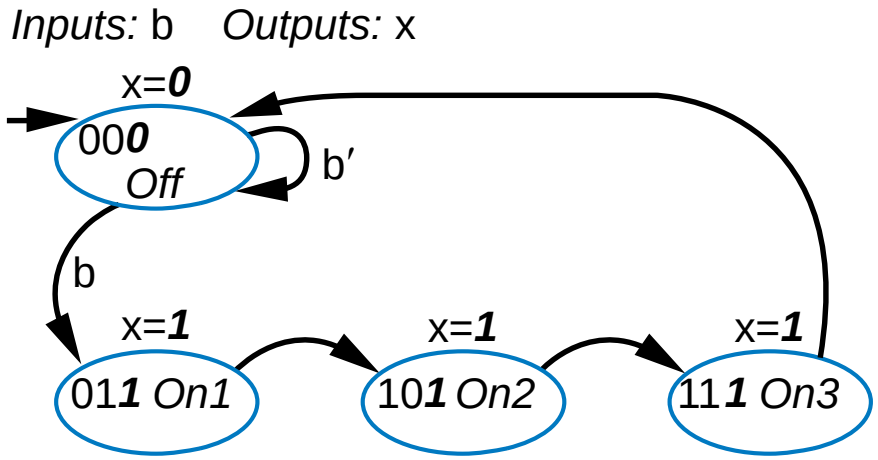
- Glitch: Temporary values on outputs that appear soon after input changes, before stable new output values
- Designer must determine whether glitching outputs may pose a problem
  - If so, may consider adding flip-flops to outputs
    - Delays output by one clock cycle, but may be OK
    - Called *registered* output

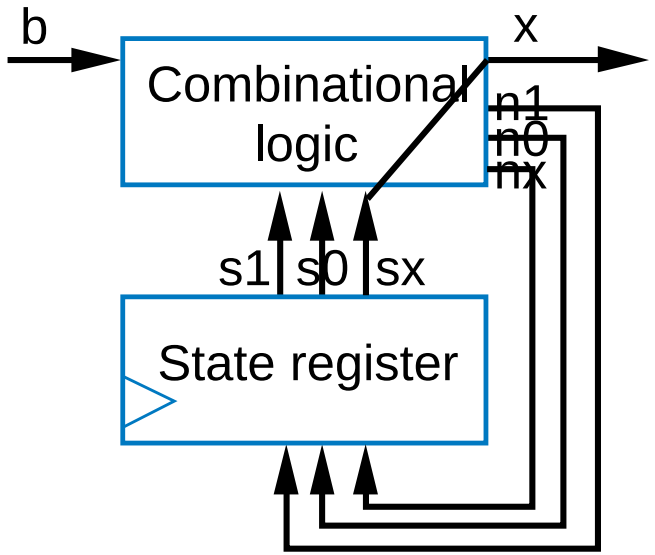


Laser timer controller with flip-flop to prevent glitches on x from unintentionally turning on laser

Glitching

- Alternative registered output approach, avoid 1 cycle delay:
  - Add extra state register bit for each output
  - Connect output directly to its bit
  - No logic between state register flip-flop and output, hence no glitches





But, uses more flip-flops, plus more logic to compute next state